Milestone 2 — Multi-Object Detection

Study Group 9

73 object classes (OBJ001–OBJ073) · YOLOv8s detector · composites (2–4 objects) and grid images (4–25 objects)

Contents

1. Introduction 2

2. Dataset 2

2.1 Shared single-object photos . . . 2

2.2 Cleaning and preprocessing . . . 2

2.3 Photo-level split . . . 2

2.4 Automatic object boxes . . . 2

2.5 Multi-object composites (2–4 objects) . . . 3

2.6 Grid images (4–25 objects) . . . 3

3. Detector Architecture and Training 3

3.1 Architecture . . . 4

3.2 Three-stage training . . . 4

4. Testing and Evaluation 4

4.1 Protocol and metrics . . . 4

4.2 Detector on the composites (2–4 objects) . . . 5

4.3 Detector on the grid images (4–25 objects) . . . 5

5. Live Demonstration 6

6. Results and Limitations 7

7. Reproducibility 8

Appendix A. Full Training Configuration 9

Appendix B. Additional Live-Demo Detections 9

Appendix C. Additional Test-Set Detections 10

Appendix D. Object IDs (73 classes) 11

1. Introduction

Milestone 2 extends the single-object classifier of Milestone 1 to **object detection**: given an image containing several objects, the system must report the **Object ID and bounding-box location of every object**. Following the project specification, we build at least 100 multi-object images by concatenating randomly-selected single-object photos, use **transfer learning from YOLOv8** (the multi-object dataset is small relative to a detection model), and compare and evaluate before selecting a model for the demonstration. We cover all 73 classes (OBJ001–OBJ073); YOLO class indices 0–72 map to these IDs in order.

Two multi-object datasets are built from the shared single-object photos — free-form **composites** of two to four objects, and full **grid images** of four to twenty-five objects in the class's grid format. A single YOLOv8s detector is trained in three stages and evaluated on held-out test splits, and a live demonstration (Section 5) runs the deployed detector on unseen grid images.

2. Dataset

2.1 Shared single-object photos

Every single-object image is a 224×224 colour JPEG named `OBJxxx_nnn.jpg` in a per-object folder. The shared set holds 73 classes; 70 classes also contain *background-only* photos (marked _bg_) that show the surroundings without the object and serve as negative examples for the detector.

2.2 Cleaning and preprocessing

Each file is opened and verified, EXIF orientation corrected, transparency composited on white, converted to RGB, resized with Lanczos filtering preserving aspect ratio, padded with white to 224×224 and saved as JPEG quality 95. Before splitting, **116 exact duplicates** (identical standardized pixels or identical output JPEG, by SHA-256) were removed so no photo appears in two splits, leaving **7,370 images: 7,000 object photos (89–140 per class) and 370 background-only**.

2.3 Photo-level split

A class-stratified split (seed 42) assigns roughly 70/15/15% of photos to train/validation/test. The split manifest is saved and reused by every later stage; validation is used only for checkpoint and model selection and test only for the final evaluation. Composites and grids draw tiles only from photos of their own split, so no source photo crosses splits.

Table 1. Images per split at each stage of the data pipeline.

Data (stage)	Train	Val	Test	Total
Cleaned photos	5,160	1,105	1,105	7,370
Photos with an auto box	4,735	1,019	995	6,749
Composites (2–4 obj, 640 px)	7,000	1,500	1,500	10,000
— object boxes	20,975	4,521	4,492	29,988
Grid images (4–25 obj)	3,500	750	750	5,000
— object boxes	51,800	11,100	11,100	74,000

2.4 Automatic object boxes

The shared photos carry only class labels (bare IDs), so each cleaned object photo is labelled automatically with one box. A prompt-free open-vocabulary detector (**YOLOE-26**) proposes candidates; an **EfficientNet-B0** classifier (from Milestone 1) scores the known class and **DINOv2** features measure how strongly a crop recurs across the class (co-localisation); a selection step chooses the best box, and flagged photos get a text-prompted pass and a manual review. Of 7,000 object photos, 6,705 boxes came from the prompt-free pass, 264 from the text pass, 23 were hand-drawn and 6 confirmed; **251 photos (3.6%) were excluded** (249 unreviewed high-risk, 2 with no object), leaving 6,749 boxed photos plus 370 background photos with empty labels. The median box covers 14.6% of the image.

2.5 Multi-object composites (2–4 objects)

10,000 composites concatenate two to four whole cleaned photos of distinct classes (3,295 / 3,422 / 3,283 with 2 / 3 / 4 objects) in six layouts scaled to a 640-px long side, giving 29,988 boxes. Each tile is mirrored with probability 0.5, 4,555 images receive ±6% brightness/contrast jitter, and 984 carry one extra background-only tile with no box. Class/photo decks balance draws (every class in 409–412 images).

Table 2. Composite layouts and image sizes.

Composite layout	Image size (w×h)	Images
Two columns (1×2)	640 × 320	2,060
Two rows (2×1)	320 × 640	753
Three tiles, asym. horizontal	640 × 427	1,674
Three tiles, asym. vertical	427 × 640	1,728
Four tiles (2×2)	640 × 640	3,785
Total	—	10,000

2.6 Grid images (4–25 objects)

The class's grid format places 224×224 photos side by side in a full grid. We build **5,000** grid images, 1,000 per grid size (700/150/150 per split), each cell one photo at its own 224×224 size without resizing and all objects of distinct classes. Tiles are mirrored with probability 0.5 and 2,245 images get the mild photometric jitter; there are no background tiles because every cell is filled. This yields **74,000 boxes**, with every class appearing 1,013–1,015 times.

Table 3. Grid sizes.

Grid	Objects / image	Image size (px)	Images
2×2	4	448 × 448	1,000
3×3	9	672 × 672	1,000
4×4	16	896 × 896	1,000
5×4	20	1120 × 896	1,000
5×5	25	1120 × 1120	1,000
Total	—	—	5,000

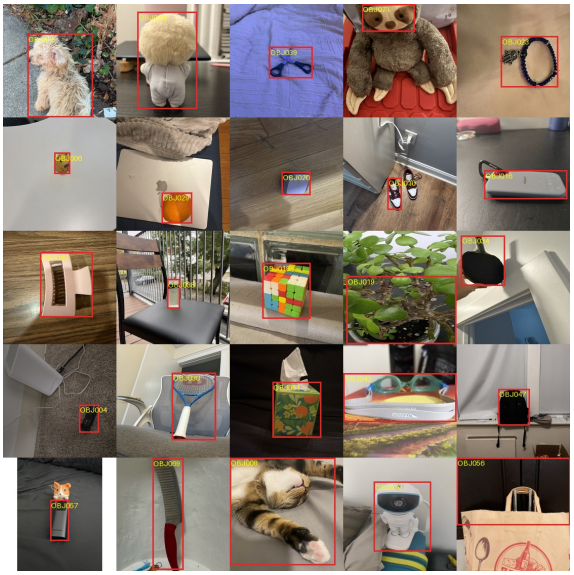


Figure 1. A generated 5×5 grid image with ground-truth boxes and Object IDs.

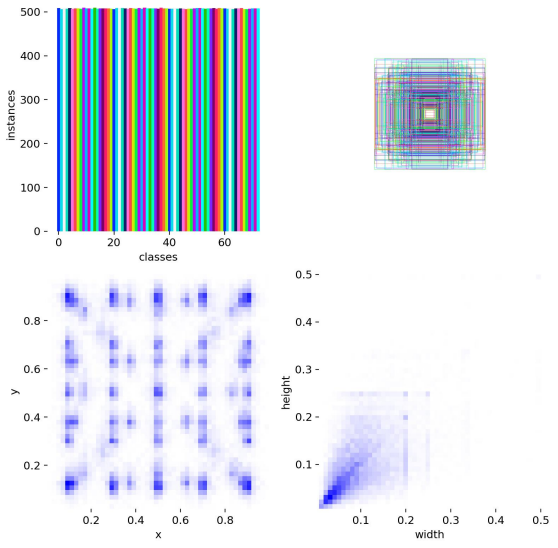


Figure 2. Training-set label statistics: class balance and box centre / size distributions.

3. Detector Architecture and Training

3.1 Architecture

As required, the detector uses transfer learning: **YOLOv8s** (Ultralytics) initialized from COCO-pretrained weights. YOLOv8 is a one-stage, anchor-free detector — a CSP-style backbone with C2f blocks, a path-aggregation neck fusing features at strides 8/16/32, and a decoupled head predicting class scores (binary cross-entropy) and box distances (CIoU + distribution-focal loss) at every location of three feature maps. The head was rebuilt for the 73 classes; 349 of 355 weight tensors transferred and all layers were fine-tuned. The model has **11.16 M parameters** and 28.8 GFLOPs at 640 px. YOLOE and DINOv2 are annotation helpers only; the trained detection model is YOLOv8s.

3.2 Three-stage training

The detector is trained in three stages (GPU, seed 20261003, deterministic). All use Ultralytics' automatic optimizer (AdamW, lr 0.00013, momentum 0.9, weight decay 0.0005), three warm-up epochs, cosine decay, mixed precision and online augmentation (mosaic, HSV, translation, scaling, flips), selecting the checkpoint with the highest validation mAP50-95. **Stage 1** trains on the 7,000 composites; **Stage 2** adds the 4,978 single-object training photos (mosaic halved); **Stage 3** fine-tunes on the 3,500 grid images at **1,120 px** — the largest grid's size, so no grid is shrunk — and is the deployed detector.

Table 4. Detector training stages.

Setting	Stage 1	Stage 2	Stage 3 (deployed)
Initialization	COCO YOLOv8s	COCO YOLOv8s	stage-2 best.pt
Training images	7,000 composites	7,000 comp. + 4,978 photos	3,500 grid images
Validation	1,500 composites	1,500 composites	750 grid images
Input / batch	640 px / 16	640 px / 16	1,120 px / 8
Mosaic share	1.0	0.5	1.0
Epochs run / selected	36 / 26	39 / 29	11 / 3
Val mAP50-95	0.798	0.809	0.789 (grid val)

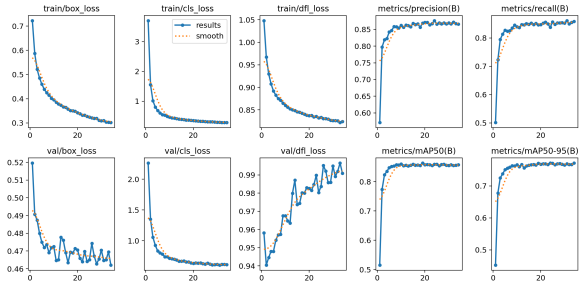


Figure 3. Training and validation curves of the grid-detection run (losses; validation P, R, mAP50, mAP50-95).



Figure 4. A training batch showing online augmentation (mosaic, colour jitter, flips).

4. Testing and Evaluation

4.1 Protocol and metrics

All numbers are on held-out **test** data that played no part in training or selection. Detectors are scored with precision, recall, **mAP50** and **mAP50-95** (mean AP over IoU 0.50–0.95). Because average precision does not say whether *every* object is found, we also measure the task directly at a fixed confidence of 0.25 with class-agnostic NMS at IoU 0.5: a prediction is correct when it has the right ID and IoU ≥ 0.5 with a not-yet-matched ground-truth box. From the matches we report object precision/recall/F1, the share of images whose predicted ID set equals the truth, and the

share in which every object is found with no extra box.

4.2 Detector on the composites (2–4 objects)

Evaluation uses the 1,500 composite test images (4,492 objects); Stage 3 is run at 1,120 px as in the application. Adding single-object photos (stage 2) raised test mAP50-95 from 0.785 to 0.795; wrong IDs are rare (0.6% of objects) and correct detections reach a mean IoU of 0.94, so found boxes are tight.

Table 5. Detection metrics on the composite test images.

Metric	Stage 1	Stage 2	Stage 3 (1120 px)
Precision	0.870	0.879	0.872
Recall	0.874	0.876	0.863
mAP50	0.869	0.878	0.871
mAP50-95	0.785	0.795	0.781
Object F1 (conf 0.25)	0.874	0.870	0.866
Images every object correct	59.7%	59.1%	55.9%

4.3 Detector on the grid images (4–25 objects)

The 750 grid test images contain 11,100 objects. The stage-2 detector (trained on 640-px composites, never on grids) already transfers when run at 1,120 px, where the 224-px tiles appear at 224–560 px — the range seen in training — so grid fine-tuning (stage 3) changes little.

Table 6. Detection on the grid test images at 1,120 px.

Metric	Stage 2	Stage 3 (deployed)
mAP50	0.877	0.877
mAP50-95	0.789	0.782
Precision / recall	0.881 / 0.868	0.891 / 0.864
Object F1 (conf 0.25)	0.875	0.877
Correct set of IDs	32.1%	35.1%
Images every object correct	16.9%	16.9%

Table 7. Per-grid results of the deployed detector (150 test images per grid).

Grid	Objects	Object F1	Correct ID set	Every object correct
2×2	4	0.866	61.3%	48.0%
3×3	9	0.883	46.7%	20.7%
4×4	16	0.883	36.0%	10.0%
5×4	20	0.879	23.3%	4.7%
5×5	25	0.871	8.0%	1.3%

Object-level accuracy is essentially flat across grids (object F1 0.87–0.88) — the detector does not degrade as objects get smaller and more numerous — but whole-image success falls steeply, exactly as independent per-object errors predict (if each object is found with probability ≈0.87, all n are found with probability $\approx 0.87^n$). For large grids, object recall and precision are the meaningful targets.

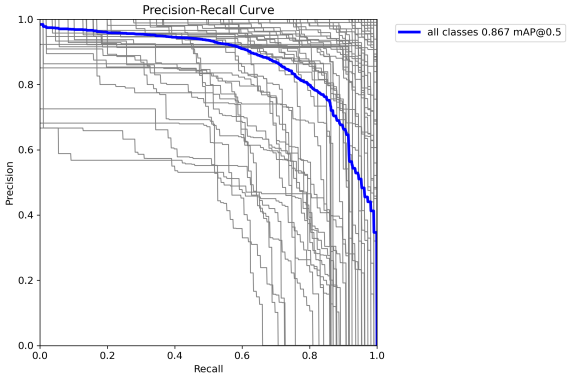


Figure 5. Precision–recall curve on the grid test set (all classes).

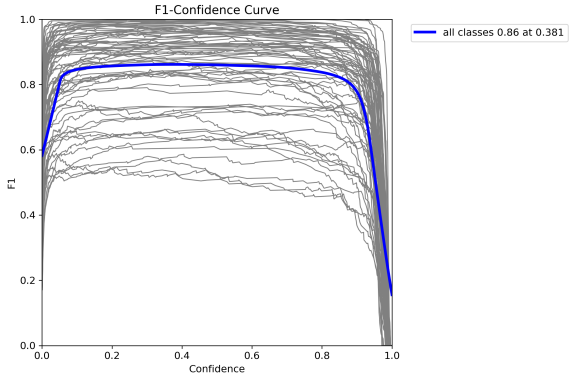


Figure 6. F1 vs. confidence on the grid test set.



Figure 7. Detector predictions on a test batch (ID, confidence).



Figure 8. Ground-truth labels for the same batch.

5. Live Demonstration

To demonstrate the deployed detector on unseen inputs, we ran the application (`run_live_demo.py`) on the **ten grid images prepared for the TA** — two per grid size (2×2 … 5×5), none in any training or test split. For each image it prints every detected Object ID with its box and confidence and saves an annotated copy to `live_demo/Run_X/outputs/`. Across the ten images it detected **146 of the 148 objects**, most with confidence above 0.9; the 2×2, 3×3, 5×4 and 5×5 grids were essentially complete, with only one missed object on a dense grid and an occasional duplicate box.

Table 8. Live-demo detection counts on the ten TA grid images.

Grid image	Objects	Detected	Grid image	Objects	Detected
grid_2x2_1	4	4	grid_4x4_2	16	17
grid_2x2_2	4	4	grid_5x4_1	20	20
grid_3x3_1	9	9	grid_5x4_2	20	17
grid_3x3_2	9	10	grid_5x5_1	25	24
grid_4x4_1	16	16	grid_5x5_2	25	25
			Total	148	146



Figure 9. Live demo — 5×5 grid (25 objects), all detected.



Figure 10. Live demo — 4×4 grid (16 objects).



Figure 11. Live demo — 2×2 grid (4 objects).



Figure 12. Live demo — 3×3 grid (9 objects).

6. Results and Limitations

Both required functions are met. The fine-tuned YOLOv8s detector finds about **87% of objects with the correct ID and a box overlapping the truth by at least 50%**, in composites of two to four photos and in grids of up to twenty-five, and almost never assigns a wrong ID to an object it has located; its errors are loose boxes and misses in roughly equal parts.

Table 9. Summary of the delivered detection models.

Requirement	Model / file	Test result
Multi-object ID and location	YOLOv8s stage 3 (deployed) `best.pt`	Grids: mAP50 0.877, mAP50-95 0.782, 87.4% of objects found. Composites: mAP50-95 0.781, 86.5% found
Alternative detector	YOLOv8s stage 2 `best.pt`	Composites: mAP50 0.878, 87.6% found. Grids (1120 px): mAP50-95 0.789, 87.9% found
Demonstration	`run_live_demo.py`	10 TA grid images: 146 / 148 objects detected

Key findings. Transfer learning was decisive — the COCO-pretrained detector reached most of its accuracy within five epochs on synthetic data. Showing the detector the kind of image it will see mattered when that kind was new

(single-object photos, stage 2) but hardly at all for grids (stage 3), which are only a rearrangement of known tiles: it was enough to run the detector at a size where the tiles appear at roughly the scale seen in training. Object-level accuracy is stable across all grid sizes; only whole-image success falls with object count, the expected consequence of independent per-object errors.

Limitations. Boxes come from an automatic pipeline; although checked on a sample, a small fraction may be loose or wrong, which caps achievable mAP and affects a few low-recall classes. Grid cells are clean and non-overlapping, so the detector is not tested on occlusion or arbitrary placement within one scene. Future work: add genuinely multi-object photographs, tighten labels by requiring agreement between two detectors, and sweep input size and model scale.

7. Reproducibility

Code: **https://github.com/Sneha0504/ie7615-object-identification**. Datasets and weights are large and are not committed. Pipeline: `annotate_single_object.py` (per-object boxes) → `generate_multi_object.py` / `generate_grid_dataset.py` (composites / grids) → `train_yolo_detector.py` (training, on GPU) → `run_live_demo.py` (detect objects in any new image, printing each Object ID with its box and saving annotated outputs). Seeds are fixed throughout.

Appendix A. Full Training Configuration

Ultralytics settings shared by the three detector stages (input size and batch vary by stage — see Table 4). Other options are left at library defaults.

Table 10. Training hyperparameters.

Setting	Value	Setting	Value
Base weights	yolov8s.pt	Optimizer	AdamW (auto)
Initial LR (lr0)	0.00013	Final LR factor	0.01
Momentum	0.937	Weight decay	0.0005
Warm-up epochs	3	LR schedule	cosine
Epochs / patience	50 / 10	Mixed precision	on (GPU)
Input size	640 / 1,120 px	Batch	16 / 8
Box / cls / dfl gains	7.5 / 0.5 / 1.5	Mosaic	1.0 (0.5 st. 2)
HSV (h / s / v)	0.015 / 0.7 / 0.4	Translate / scale	0.1 / 0.5
Flip (left-right)	0.5	Seed / deterministic	20261003 / yes

Appendix B. Additional Live-Demo Detections

Further annotated outputs from `run_live_demo.py` on the remaining TA grid images (predicted Object ID and confidence on each object).



Figure B1. Live demo — 2×2 grid (4 objects).

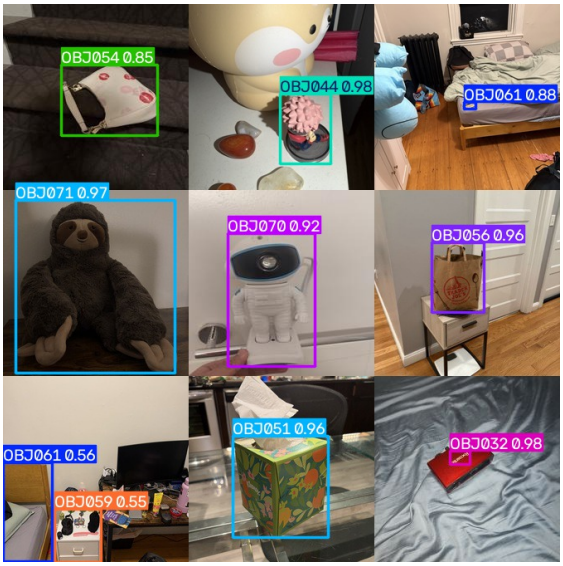


Figure B2. Live demo — 3×3 grid (9 objects).

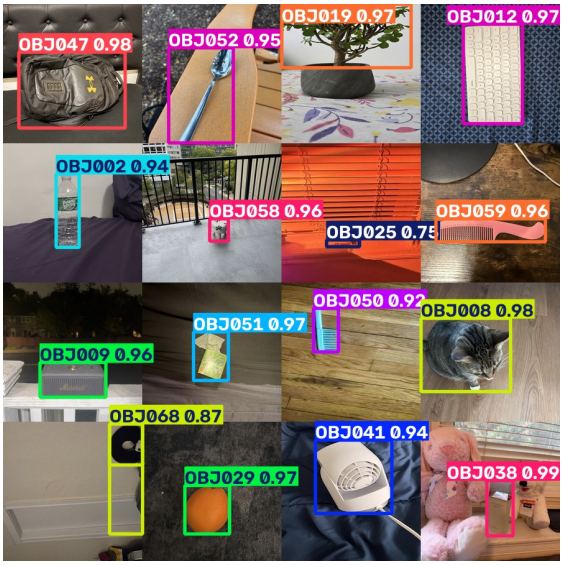


Figure B3. Live demo — 4×4 grid (16 objects).



Figure B4. Live demo — 5×4 grid (20 objects).

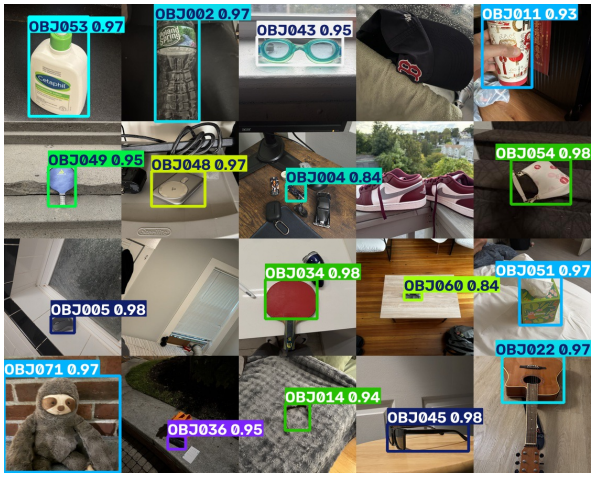


Figure B5. Live demo — 5×4 grid (20 objects).



Figure B6. Live demo — 5×5 grid (25 objects).

Appendix C. Additional Test-Set Detections

Predictions of the deployed detector on held-out grid test images (predicted Object ID and confidence on each object).

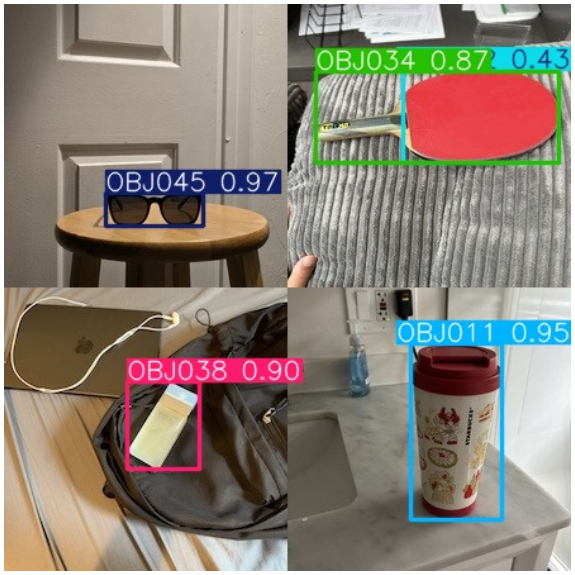


Figure C1. Detector on a grid test image.

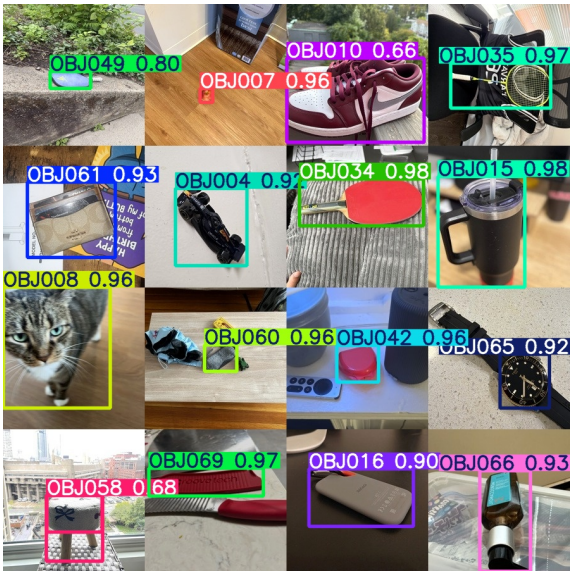


Figure C2. Detector on a grid test image.

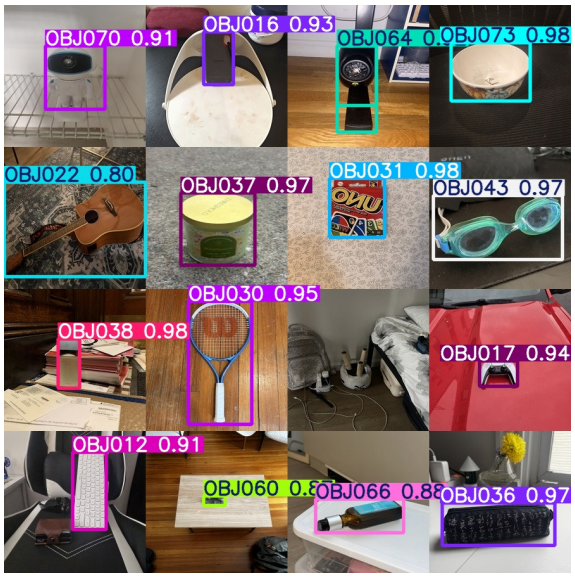


Figure C3. Detector on a grid test image.



Figure C4. Detector on a grid test image.

Appendix D. Object IDs (73 classes)

YOLO class indices 0–72 map to OBJ001–OBJ073 in order:

OBJ001	OBJ002	OBJ003	OBJ004	OBJ005	OBJ006	OBJ007	OBJ008
OBJ009	OBJ010	OBJ011	OBJ012	OBJ013	OBJ014	OBJ015	OBJ016
OBJ017	OBJ018	OBJ019	OBJ020	OBJ021	OBJ022	OBJ023	OBJ024
OBJ025	OBJ026	OBJ027	OBJ028	OBJ029	OBJ030	OBJ031	OBJ032
OBJ033	OBJ034	OBJ035	OBJ036	OBJ037	OBJ038	OBJ039	OBJ040
OBJ041	OBJ042	OBJ043	OBJ044	OBJ045	OBJ046	OBJ047	OBJ048
OBJ049	OBJ050	OBJ051	OBJ052	OBJ053	OBJ054	OBJ055	OBJ056
OBJ057	OBJ058	OBJ059	OBJ060	OBJ061	OBJ062	OBJ063	OBJ064
OBJ065	OBJ066	OBJ067	OBJ068	OBJ069	OBJ070	OBJ071	OBJ072
OBJ073							